

Multi-Agent Architectures for LLM-Based Code Generation: Evaluating Adaptive Routing and Model Specialization

Ivan Necerini
s345147

Jacopo Rialti
s346357

Emanuele Romano
s346325

Marco Donatucci
s337624

Ferdinando Del Vecchio
s347426

Abstract

Single-prompt code generation with Large Language Models (LLMs) frequently fails on tasks requiring multi-step reasoning or edge-case handling. Multi-agent architectures that decompose development into specialized roles (planning, implementation, testing, and review) offer a principled approach to iterative refinement, yet their empirical benefits remain under-explored.

We conduct a controlled comparison of four LLM-based code generation architectures on the HumanEval benchmark (164 tasks): a single-agent baseline, a multi-agent pipeline using one model across all roles, a multi-agent system with tiered models (1.5B–32B parameters) and difficulty-based routing, and an always-large variant. We further evaluate prompt repetition, a technique that duplicates user queries to improve token attention.

Multi-agent orchestration achieves 72.6% pass rate versus 67.7% for single-agent (+4.9 pp), with the review loop recovering 22% of initially failing tasks. However, adaptive routing underperforms the always-large strategy (62.2% vs 81.7%), as the smallest model (1.5B) consistently fails and triggers costly escalation cascades. Prompt repetition degrades multi-agent accuracy by up to 7.9 pp, likely due to context dilution in already-rich conversation histories.

These results demonstrate that multi-agent pipelines improve correctness when paired with sufficiently capable models, but adaptive routing provides no benefit when tier-S models fall below a reliability threshold. The findings inform the design of cost-effective LLM orchestration systems.

single-prompt interactions often fail on complex development tasks that require multi-step reasoning, iterative refinement, or careful handling of edge cases. This limitation has motivated the exploration of multi-agent architectures that decompose the software development process into specialized roles, potentially improving both code quality and functional correctness.

Multi-agent systems for code generation typically assign distinct responsibilities to different agents (such as planning, implementation, testing, and review) mirroring established software engineering practices. While this approach is conceptually appealing, the empirical benefits of such architectures remain underexplored. Key questions include whether the overhead of agent coordination is justified by quality improvements, whether specialized models for each role outperform a single model serving all roles, and whether adaptive strategies can reduce computational costs without sacrificing correctness.

This work presents a systematic empirical comparison of LLM-based code generation architectures, ranging from single-agent baselines to multi-agent pipelines with adaptive model routing. We implement and evaluate three primary architectures: (A) a single-agent baseline, (B) a multi-agent pipeline using a single model for all roles, and (C) a multi-agent pipeline with specialized models per role and adaptive routing based on task difficulty estimation. Additionally, we investigate the effect of prompt repetition, a recently proposed technique for improving non-reasoning LLMs (Leviathan et al., 2025), on code generation accuracy across all architectures.

Our evaluation is conducted on the HumanEval benchmark (Chen et al., 2021), comprising 164 hand-written programming problems with assertion-based tests. We address the following research questions:

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation, enabling developers to produce functional software from natural language specifications. However,

- **RQ1:** Does a multi-agent pipeline with role separation (Planner, Developer, Tester, Reviewer) improve functional correctness compared to a single-agent approach?
- **RQ2:** Do specialized models assigned to each role provide measurable benefits over using a single model for all roles?
- **RQ3:** Can adaptive routing, selecting developer model capacity based on estimated task difficulty, reduce computational cost while maintaining or improving quality?
- **RQ4:** Does prompt repetition improve code generation accuracy for the models used in this study?

The contributions of this work include: (1) a reproducible multi-agent framework for code generation using LangGraph, (2) an empirical comparison of single-agent versus multi-agent architectures on standardized benchmarks, (3) an analysis of adaptive model routing based on Scrum-style story point estimation, and (4) an evaluation of prompt repetition effects in the context of code generation.

2 System Overview

This section presents the architecture of our multi-agent code generation system, describing the logical pipeline, difficulty estimation mechanism, escalation policy, and the experimental configurations evaluated in this study.

2.1 Multi-Agent Pipeline Architecture

The proposed multi-agent pipeline decomposes the code generation task into five specialized components, each responsible for a distinct phase of the development process. The pipeline follows the sequence: **Planner** → **Router** → **Developer** → **Tester** → **Reviewer**, with conditional looping upon test failure.

The **Planner** agent receives the task specification and estimates its difficulty using Scrum-style story points, providing a rationale for the assessment. The **Router** uses this difficulty estimate to select an appropriately-sized Developer model, balancing computational cost against task complexity. The **Developer** agent generates the implementation code based on the task description and, in retry iterations, incorporates feedback from previous failures. The **Tester** is a non-LLM component

that executes the generated code against assertion-based tests in a sandboxed environment. Finally, the **Reviewer** agent analyzes the test results and provides actionable feedback for subsequent iterations without modifying the code directly.

Figure 1 illustrates the four architectural configurations evaluated in this study, highlighting the differences in pipeline structure and model allocation.

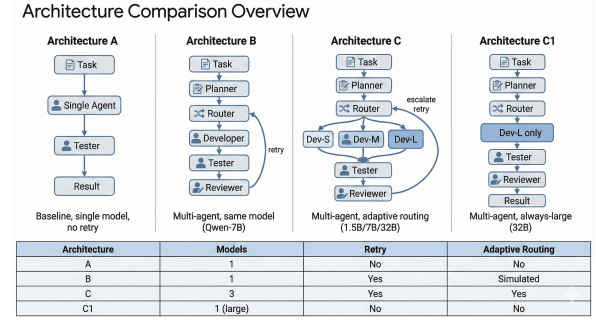


Figure 1: Comparison of the four experimental architectures: (A) single-agent baseline, (B) multi-agent with single model, (C) multi-agent with adaptive routing across three model tiers, and (C1) multi-agent always using the largest model.

2.2 Difficulty Estimation via Story Points

Task difficulty estimation is performed by the Planner agent using a Fibonacci-based story point scale: {1, 2, 3, 5, 8}. This scale, borrowed from Agile software development practices, serves as a routing signal rather than a time estimate. The Planner evaluates each task along five dimensions: (1) algorithmic complexity, (2) edge case density, (3) external constraints such as performance requirements or strict I/O formatting, (4) coupling with other components, and (5) specification ambiguity.

Story points are mapped to developer tiers as follows: points 1–2 indicate trivial to small tasks routed to the smallest model (Tier S), points 3–5 represent medium to challenging tasks routed to a mid-sized model (Tier M), and point 8 denotes hard tasks requiring the largest model (Tier L). This mapping is visualized in Figure 2.

2.3 Adaptive Routing and Escalation Policy

The system implements an adaptive escalation policy that progressively increases model capacity upon test failure. When the Tester reports a failure, the Router escalates the task to the next tier: S → M → L. A maximum of two escalations is permitted, after which the pipeline terminates regardless

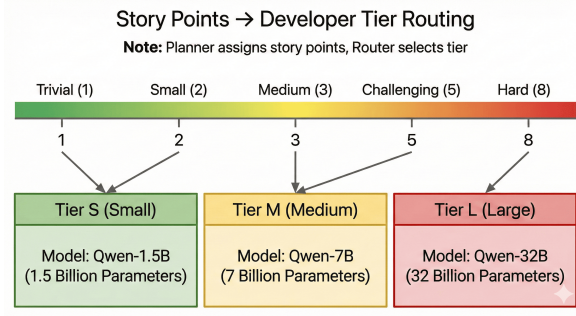


Figure 2: Mapping from story points to developer tiers. Each tier corresponds to a different model capacity: Tier S (1.5B parameters), Tier M (7B parameters), and Tier L (32B parameters).

of test outcome.

Upon escalation, the Developer receives the previous code, failure history (error messages from test execution), and feedback from the Reviewer. This accumulated context enables the stronger model to address specific deficiencies identified in prior attempts. Figure 3 illustrates the escalation flow and termination conditions.

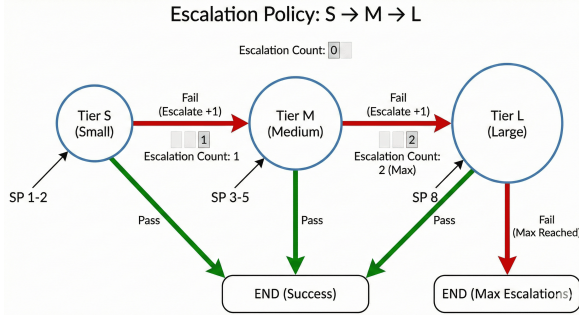


Figure 3: Escalation policy flow. On test failure, the system escalates from Tier S to M to L. The pipeline terminates upon test success or after reaching maximum escalations.

2.4 Experimental Architectures

We evaluate five architectural configurations to isolate the effects of multi-agent orchestration, model specialization, and adaptive routing:

Architecture A serves as the single-agent baseline. A single LLM receives the task and generates code in one pass without planning, review, or retry mechanisms.

Architecture B introduces the full multi-agent pipeline while keeping the model constant. All roles (Planner, Developer (all tiers), and Reviewer) use the same model. This configuration isolates the effect of role separation and iterative refinement from model capacity differences.

Architecture C employs specialized models for each role. The Planner and Reviewer use a generalist model suited for reasoning and analysis. The Developer role uses capacity-matched models: a small model (1.5B) for Tier S, a medium model (7B) for Tier M, and a large model (32B) for Tier L.

Architecture C1 is a variant of C that bypasses adaptive routing by always using the Tier L developer (32B model), regardless of the Planner’s difficulty estimate. This configuration serves as an upper-bound reference for comparing the efficiency-quality trade-off of adaptive routing.

Architecture C2 is an ablation study that removes the Tier S developer entirely. Tasks that would normally route to Tier S are instead handled by Tier M from the start. This configuration tests whether the smallest model tier is beneficial or introduces unnecessary failure cascades.

Table 1 summarizes the model assignments for each architecture.

Role	A	B	C	C1	C2
Planner	—	Q-7B	L-8B	L-8B	L-8B
Dev-S	—	Q-7B	Q-1.5B	—	—
Dev-M	—	Q-7B	Q-7B	—	Q-7B
Dev-L	—	Q-7B	Q-32B	Q-32B	Q-32B
Reviewer	—	Q-7B	L-8B	L-8B	L-8B
Baseline	Q-7B	—	—	—	—

Table 1: Model assignments per architecture. Q = Qwen2.5-Coder-Instruct, L = Meta-Llama-3-Instruct. Numbers indicate model size in billions of parameters.

2.5 Prompt Repetition Technique

We investigate the effect of prompt repetition, a technique recently proposed by Leviathan et al. (Leviathan et al., 2025) for improving non-reasoning LLM performance. The technique transforms the user prompt from $\langle \text{QUERY} \rangle$ to $\langle \text{QUERY} \rangle \backslash \text{n} \backslash \text{n} \langle \text{QUERY} \rangle$, duplicating the input with a newline separator.

The rationale is that LLMs are causal language models where each token can only attend to preceding tokens. By repeating the prompt, tokens in the second occurrence can attend to all tokens in the first occurrence, potentially improving comprehension of complex instructions. The overhead is approximately $2\times$ input tokens during the pre-fill stage, with no change to output token count. Figure 4 visualizes this attention pattern difference.

For each base architecture (A, B, C), we evaluate a corresponding prompt repetition variant (A-PR,

B-PR, C-PR), applying repetition to all user-facing prompts while leaving system prompts unchanged.

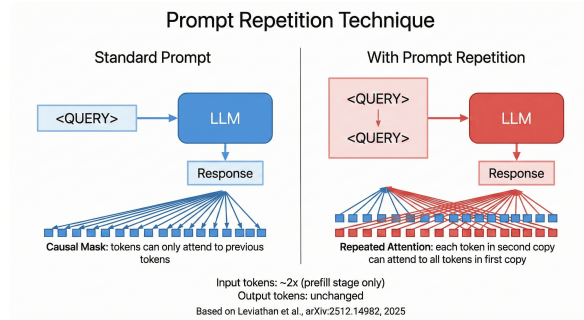


Figure 4: Comparison of attention patterns between standard prompting and prompt repetition. With repetition, tokens in the second copy can attend to all tokens in the first copy.

2.6 Implementation Details

The multi-agent pipeline is implemented using LangGraph, a framework for building stateful, graph-based LLM applications. The workflow is defined as a StateGraph where each agent corresponds to a node, and edges define the execution flow including conditional transitions based on test outcomes.

State management is handled through a TypedDict-based GraphState object that propagates task metadata, planning outputs, generated code, test results, and failure history across nodes. This structured state enables clean separation of concerns and facilitates debugging.

For execution monitoring and debugging, we integrate LangSmith, which provides tracing capabilities for inspecting individual LLM calls, token usage, and execution timelines. This tooling proved essential for diagnosing failure modes and optimizing prompt designs during development.

3 Experimental Setup

This section describes the dataset, execution environment, evaluation metrics, and experimental configurations used in our study.

3.1 Dataset

We evaluate all architectures on the HumanEval benchmark (Chen et al., 2021), a widely-used dataset for assessing code generation capabilities of large language models. HumanEval consists of 164 hand-written Python programming problems, each comprising a function signature with docstring, a

set of assertion-based tests, and a reference solution (not used during generation).

Each task is structured as follows: the prompt field contains the function signature and docstring describing the expected behavior; the test field contains Python assertions that validate the implementation; and the entry_point field specifies the function name to be implemented. This structure enables automated evaluation through assertion-based testing, where generated code is concatenated with test assertions and executed in a sandboxed subprocess with a timeout of 5 seconds.

HumanEval was chosen for several reasons: it provides a controlled benchmark with diverse problem types ranging from simple string manipulation to algorithmic challenges; it uses assertion-based testing which yields unambiguous pass/fail outcomes; and it is widely adopted in the literature, facilitating comparison with prior work.

3.2 Execution Environment

All experiments were conducted on Kaggle Notebooks, a cloud-based computational platform providing reproducible execution environments. This choice ensures that experiments can be replicated without requiring local GPU resources.

Language models were accessed through the HuggingFace Inference API, which provides serverless endpoints for model inference. This approach offers several advantages: it eliminates the need for local model deployment, provides consistent inference performance, and enables access to models of varying sizes (1.5B to 32B parameters) without hardware constraints.

3.3 Evaluation Metrics

We employ four categories of metrics to comprehensively evaluate the architectures.

Primary Metrics. The primary measure of functional correctness is the **Pass Rate**, defined as the percentage of tasks where all test assertions pass. We also report **Pass@1**, which measures correctness on the first generation attempt (for Architecture A this equals the pass rate, while for multi-agent architectures it reflects success before any retry or escalation). To quantify uncertainty, we compute **95% confidence intervals** using the Wilson score interval for binomial proportions.

Cost Metrics. To assess computational efficiency, we measure **Total Tokens** (sum of input and output tokens per task), **API Calls** (number of

LLM invocations per task), and **Execution Time** (end-to-end wall-clock time per task in seconds). These metrics enable cost-benefit analysis of multi-agent overhead.

Adaptive Metrics. For architectures employing the multi-agent pipeline (B, C, C1, C2), we track additional metrics specific to the adaptive mechanisms. **Retry Count** measures the number of Developer→Tester→Reviewer loops executed until success or final failure. **Escalation Count** tracks the number of tier transitions (S→M or M→L) triggered by test failures. **Story Point Accuracy** evaluates the correlation between the Planner’s difficulty estimate and actual task outcome. **Tier Distribution** reports the percentage of tasks routed to each developer tier (S/M/L).

Static Code Quality Metrics. Beyond functional correctness, we assess the quality of generated code using static analysis metrics computed via the Radon library.

Cyclomatic Complexity (CC), introduced by McCabe (McCabe, 1976), measures the number of linearly independent paths through a program’s control flow graph. It is computed as:

$$CC = E - N + 2P \quad (1)$$

where E is the number of edges in the control flow graph, N is the number of nodes, and P is the number of connected components (typically 1 for a single function). Each control flow statement (if, elif, for, while, except, and, or) increments the complexity by one. Following Ebert et al. (Ebert et al., 2016), we interpret CC values as: 1–5 (low risk, simple code), 6–10 (moderate risk), 11–20 (high risk, complex code), and 21+ (very high risk, untestable).

Maintainability Index (MI) is a composite metric estimating how maintainable source code is (Microsoft, 2024). It is computed as:

$$MI = \max\left(0, \frac{100 \times (171 - 5.2 \ln(HV) - 0.23 \times CC - 16.2 \ln(LOC))}{171}\right) \quad (2)$$

where HV is the Halstead Volume (based on operator and operand counts), CC is Cyclomatic Complexity, and LOC is lines of code. The index ranges from 0 to 100, where higher values indicate better maintainability: 0–19 (low), 20–39 (moderate), 40–64 (good), 65–84 (high), and 85–100 (excellent).

These metrics are computed for all generated solutions, enabling analysis of whether multi-agent

pipelines produce more maintainable code compared to single-agent approaches.

3.4 Experimental Configuration

We evaluate eight architectural configurations, as summarized in Table 1. Architectures A and A-PR represent single-agent baselines with and without prompt repetition. Architectures B and B-PR employ the full multi-agent pipeline with a single model for all roles. Architectures C and C-PR use specialized models with adaptive routing based on story points. Architecture C1 always uses the largest developer model regardless of difficulty estimate, serving as an upper-bound reference. Architecture C2 is an ablation study that removes the smallest developer tier.

Each configuration was evaluated on all 164 HumanEval tasks. For multi-agent architectures, a maximum of two escalations was permitted before terminating the pipeline. All prompt repetition variants (A-PR, B-PR, C-PR) apply the repetition technique to user prompts only, leaving system prompts unchanged.

4 Experimental Results

This section presents the results of our empirical evaluation, addressing the four research questions defined in Section 1. We analyze the impact of multi-agent orchestration, model specialization, adaptive routing, and prompt repetition on code generation performance. Additionally, we provide an in-depth analysis of escalation patterns, tier distribution, and static code quality metrics.

All results are based on the execution of 164 tasks from the HumanEval benchmark.

4.1 RQ1: Effect of Multi-Agent Architecture

We first investigate whether a multi-agent pipeline with role separation improves functional correctness compared to a single-agent baseline. We compare **Architecture A** (Single-Agent) and **Architecture B** (Multi-Agent), both using the Qwen2.5-Coder-7B-Instruct model for all roles.

Table 2 summarizes the performance metrics. Architecture B achieved a pass rate of **72.6%**, representing a significant improvement of 4.9 percentage points over the single-agent baseline (67.7%). The non-overlapping confidence intervals suggest this difference is statistically meaningful.

The Pass@1 metric for Architecture B (50.6%) refers to the success rate of the first Developer attempt before any Reviewer feedback or retry loops.

Table 2: Comparison of Single-Agent (A) vs Multi-Agent (B) Architectures.

Architecture	Pass Rate	Pass@1	95% CI	Passed/Total
A (Single-Agent)	67.7%	67.7%	[60.2, 74.4]	111/164
B (Multi-Agent)	72.6%	50.6%	[65.3, 78.8]	119/164

Notably, this is lower than Architecture A’s single-shot performance (67.7%), that obviously has no retry loops. However, the multi-agent retry mechanism (where the Reviewer analyzes test failures and provides actionable feedback for the Developer to iterate) successfully recovered an additional 22% of tasks (the difference between 72.6% final and 50.6% initial), validating the effectiveness of the self-correction loop.

Figure 5 illustrates the structural difference between the two approaches. The iterative loop mechanism in Architecture B allows recovery from initial errors that would be terminal in the single-shot Architecture A.

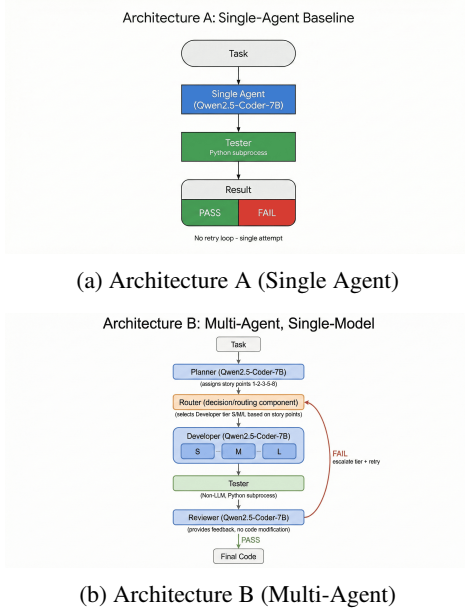


Figure 5: Structural comparison of Single-Agent vs Multi-Agent architectures. Architecture B includes Planner, Router, Developer, Tester, and Reviewer nodes with a conditional retry loop.

The cost of this improvement is reflected in execution time: Architecture B averaged 10.95 seconds per task compared to 2.98 seconds for Architecture A, representing a $3.7\times$ increase in latency. This overhead stems from the additional LLM calls (Planner, Reviewer) and potential retry iterations.

4.2 RQ2: Effect of Specialized Models

Next, we evaluate whether assigning specialized models to each role provides benefits over using a single generalist model. We compare **Architecture B** (Single-Model) with **Architecture C** (Multi-Model), where the latter uses Llama-3-8B for Planning and Reviewing, and a tiered set of Qwen models (1.5B, 7B, 32B) for development based on estimated task difficulty.

As shown in Table 3, Architecture C achieved a pass rate of **62.2%**, which is surprisingly lower than the single-model Architecture B (72.6%).

Table 3: Comparison of Single-Model (B) vs Multi-Model (C) Architectures.

Architecture	Pass Rate	95% CI	Pass@1	Avg Time (s)
B (Single-Model)	72.6%	[65.3, 78.8]	50.6%	10.95
C (Multi-Model)	62.2%	[54.6, 69.3]	35.4%	19.28

This counter-intuitive result can be attributed primarily to the performance of the smallest developer model (Tier S). In Architecture C, tasks estimated as "Easy" (Story Points 1–2) are routed to the Qwen-1.5B model. As detailed in the In-Depth Analysis (Section 4.5), tasks initially routed to Tier S consistently failed, requiring escalation through the entire tier hierarchy. The 1.5B parameter model appears insufficient for reliable code generation in this framework, even for simple tasks.

Additionally, the coordination overhead between different model families (Llama for planning/review, Qwen for development) may have introduced subtle prompt-format incompatibilities or reasoning style mismatches that hurt overall coherence.

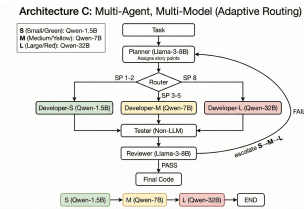


Figure 6: Architecture C: Multi-Agent with specialized models per role and adaptive tier routing.

4.3 RQ3: Effect of Adaptive Routing

To isolate the impact of adaptive routing from model specialization, we compare **Architecture C** (Adaptive S/M/L) with **Architecture C1** (Always-Large), as defined in Section 2.4. In C1, the Router

ignores story point estimates and always assigns tasks to the strongest available model (Qwen-32B).

Table 4 presents a stark contrast. Architecture C1 achieved the highest pass rate in our study at **81.7%**, significantly outperforming the adaptive version (62.2%) by 19.5 percentage points.

Table 4: Comparison of Adaptive Routing (C) vs Always-Large (C1).

Architecture	Pass Rate	Avg Time (s)	Avg API Calls	Total Time (s)
C (Adaptive)	62.2%	19.28	4.85	3163
C1 (Always-L)	81.7%	14.66	3.00	2405

Contrary to expectations, adaptive routing proved to be both less accurate *and* more expensive than the Always-Large strategy. Architecture C consumed more time (19.28s vs 14.66s) and API calls (4.85 vs 3.00) on average because the adaptive approach frequently started with weaker models, failed, and then escalated through multiple retry loops. Architecture C1, by consistently using the most capable model, solved tasks on the first attempt 81.7% of the time, avoiding costly retry overhead entirely.

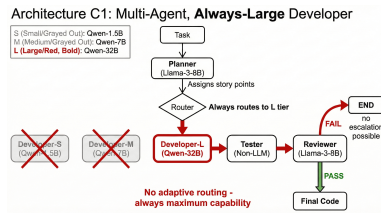


Figure 7: Architecture C1: Multi-Agent always routed to the largest (32B) developer model.

These results illustrate an interesting outcome for this specific configuration: attempting to save computational resources by using smaller models for easier tasks resulted in higher overall latency and dramatically lower success rates due to the frequency of required escalations and re-regenerations.

Ablation Study: Removing Tier S. To further investigate the role of the smallest model, we conducted an ablation study (Architecture C2, see Section 2.4) where Tier S was removed entirely. Table 5 compares the results.

Table 5: Ablation Study: Removing Tier S (C2) vs Full Adaptive (C).

Architecture	Pass Rate	Avg Time (s)	Escalations
C (With Tier S)	62.2%	19.28	152
C2 (No Tier S)	72.0%	13.32	74

Architecture C2 achieved a pass rate of **72.0%**, a substantial improvement of 9.8 percentage points over C. The removal of Tier S reduced total escalations from 152 to 74 and decreased average execution time from 19.28s to 13.32s. This confirms that the Qwen-1.5B model (Tier S) was the primary bottleneck in the adaptive configuration, its consistent failures triggered cascading escalations that consumed resources and polluted the context for subsequent attempts.

4.4 RQ4: Effect of Prompt Repetition

We evaluated the Prompt Repetition technique proposed by Leviathan et al. (Leviathan et al., 2025) across all base architectures. This technique duplicates the user prompt content to allow tokens in the second occurrence to attend to all tokens in the first occurrence, potentially improving comprehension of complex instructions.

Table 6 summarizes the effect of prompt repetition on pass rates.

Table 6: Effect of Prompt Repetition on Pass Rates.

Base Architecture	Standard	With -PR	Delta
A (Single-Agent)	67.7%	68.3%	+0.6%
B (Multi-Agent, Single-Model)	72.6%	66.5%	-6.1%
C (Multi-Agent, Multi-Model)	62.2%	54.3%	-7.9%

Our findings diverge from those reported in the original paper. While the single-agent baseline (A) showed a negligible improvement (+0.6%), the multi-agent architectures (B and C) experienced significant performance degradation (-6.1% and -7.9%, respectively).

A plausible explanation is that prompt repetition increases input context length substantially. In multi-agent pipelines, the conversation history already accumulates Planner outputs, previous code attempts, test error messages, and Reviewer feedback. Duplicating the user prompt in this already-rich context may introduce noise and make it harder for the model to attend to the most critical, recent instructions; particularly the specific feedback from the Reviewer explaining why the previous attempt failed.

This suggests that prompt repetition, while beneficial for simpler single-turn tasks, may be counterproductive in complex, multi-turn agentic workflows where context management is critical.

4.5 In-Depth Analysis

Escalation Patterns. Table 7 presents the distribution of escalation counts and their corresponding pass rates across architectures.

Table 7: Escalation Analysis by Architecture.

Arch	0-Esc	0-Esc Pass%	1-Esc	1-Esc Pass%	2-Esc	2-Esc Pass%
B	83	100%	53	62.3%	28	10.7%
C	59	98.3%	58	31.0%	47	55.3%
C1	164	81.7%	0	—	0	—

In Architecture B, tasks that passed without escalation achieved 100% success, but success dropped sharply with each escalation level (62.3% at 1-Esc, 10.7% at 2-Esc). Architecture C exhibited a similar pattern but with an unusual recovery at 2-Esc (55.3%), suggesting that the strongest Tier L model (32B) could rescue some tasks that Tier M could not.

Developer Tier Distribution. Table 8 shows the final tier assignment for each architecture.

Table 8: Developer Tier Distribution (Final Assignment).

Architecture	Count (%)			Pass Rate at Tier		
	S	M	L	S	M	L
B	57 (35%)	53 (32%)	54 (33%)	100%	100%	16.7%
C	0 (0%)	57 (35%)	107 (65%)	—	100%	42.1%
C1	0 (0%)	0 (0%)	164 (100%)	—	—	81.7%

In Architecture C, no tasks completed at Tier S, all initially routed S-tier tasks escalated. This confirms that the Qwen-1.5B model is inadequate for this code generation framework. Notably, the Tier L pass rate in Architecture C (42.1%) is much lower than in C1 (81.7%), indicating that accumulated errors from prior failed attempts may have polluted the context for subsequent generations.

Story Point Accuracy. The Planner’s story point estimates demonstrated meaningful predictive validity. In Architecture C1, which uses the same large model for all tasks regardless of estimated difficulty, tasks assigned SP=1–2 (trivial) achieved 86–100% pass rates, while SP=8 (hard) tasks achieved only 50%. This correlation confirms that the Planner correctly identifies task difficulty, as genuinely harder problems yield lower success rates even when processed by the strongest available model.

However, a separate concern emerges when comparing Architecture A’s Pass Rate (67.7%) with Architecture B’s Pass@1 (50.6%). In Architecture A, the model receives the raw HumanEval task description directly. In Architecture B, the same

task description is first processed by the Planner, which adds story point estimates and structured reasoning before forwarding it to the Developer. Despite this additional context, B’s first-attempt success rate is substantially lower than A’s single-shot performance. This suggests that the Planner’s augmented output may introduce noise or dilute the original task specification, making it harder for the Developer to focus on the core problem. While the multi-agent pipeline ultimately recovers through its retry mechanism (reaching 72.6% final pass rate), the Planner’s contribution to first-attempt success appears counterproductive.

4.6 Static Code Quality Analysis

We analyzed the quality of generated code using Cyclomatic Complexity (CC) and Maintainability Index (MI), computed via the Radon library.

Table 9: Static Code Quality Metrics by Architecture.

Architecture	Overall		Passed vs Failed	
	Avg CC	Avg MI	Passed CC	Failed CC
A (Single)	3.59	75.0	3.67	3.21
B (Multi, Single-Model)	3.87	84.5	3.76	4.66
C (Multi, Multi-Model)	3.84	85.4	4.02	3.08
C1 (Always-L)	4.58	81.1	4.55	4.85

Several observations emerge from Table 9:

- **Multi-agent architectures produce more maintainable code.** Architectures B and C achieved MI scores above 84, compared to 75 for the single-agent baseline. This suggests the review process encourages cleaner code structures.
- **The strongest model produces more complex code.** Architecture C1’s average CC (4.58) was highest, potentially reflecting more sophisticated algorithmic solutions.
- **Failed tasks in Architecture B show higher complexity.** The CC for failed tasks (4.66) exceeds that of passed tasks (3.76), indicating the model struggled with inherently complex problems.

All architectures maintained CC values in the "low risk" range (1–5), and MI values above 75 indicate "high maintainability" according to standard thresholds.

5 Conclusion

This work presented a systematic empirical comparison of LLM-based code generation architectures,

investigating the effects of multi-agent orchestration, model specialization, adaptive routing, and prompt repetition on functional correctness and code quality.

Summary of Contributions. We developed a reproducible multi-agent framework using LangGraph that implements role separation across Planner, Developer, Tester, and Reviewer agents. We conducted a comprehensive evaluation on the HumanEval benchmark comparing single-agent baselines with multi-agent pipelines of varying complexity. We introduced an adaptive routing mechanism based on Scrum-style story point estimation, and evaluated the prompt repetition technique in the context of code generation.

Key Findings. Our experiments yielded several insights. First, multi-agent orchestration with an iterative review loop improves functional correctness over single-agent approaches, achieving a 4.9 percentage point improvement (72.6% vs 67.7%) when using the same underlying model. The self-correction mechanism successfully recovered 22% of initially failing tasks. Second, model specialization with adaptive routing underperformed compared to a single-model configuration when the smallest tier model (1.5B parameters) proved insufficiently capable, causing cascading failures and context pollution. Third, the always-large strategy (Architecture C1) achieved the highest pass rate (81.7%) while simultaneously reducing computational cost, as it avoided the retry overhead incurred by adaptive routing. Fourth, prompt repetition, contrary to findings in other domains, degraded multi-agent performance by up to 7.9 percentage points, likely due to increased context length interfering with the model’s ability to focus on recent, critical feedback.

Limitations. This study has several limitations. Our evaluation is conducted on a single benchmark (HumanEval), and results may not generalize to other datasets with different problem distributions or longer code requirements. The set of models evaluated (Qwen and Llama families) represents a specific point in the rapidly evolving LLM landscape; more capable small models may validate the adaptive routing hypothesis. Story point thresholds and prompt designs were not extensively tuned, leaving potential for optimization. Finally, the execution environment (HuggingFace Inference API) introduces variability that may affect reproducibil-

ity.

Future Work. Several directions merit further investigation. Evaluating on additional benchmarks such as MBPP, APPS, or SWE-bench would assess generalization. Testing with more capable small models could validate whether adaptive routing becomes beneficial when all tiers exceed a capability threshold. Alternative difficulty estimation mechanisms, such as fine-tuned classifiers or ensemble methods, may improve routing accuracy. Dynamic prompt engineering that adapts to accumulated context length could address the prompt repetition degradation. Finally, incorporating human feedback in the review loop may further improve code quality beyond what automated testing can achieve.

References

- Mark Chen and 1 others. 2021. [Evaluating large language models trained on code](#). *arXiv preprint arXiv:2107.03374*.
- Christof Ebert, James Cain, Giuliano Antoniol, Steve Counsell, and Philip Laplante. 2016. Cyclomatic complexity. Technical report, IEEE Software. White Paper.
- Yaniv Leviathan and 1 others. 2025. [Prompt repetition improves non-reasoning llms](#). *arXiv preprint arXiv:2512.14982*.
- Thomas J. McCabe. 1976. A complexity measure. *IEEE Transactions on Software Engineering*, SE-2(4):308–320.
- Microsoft. 2024. Code metrics - maintainability index range and meaning. Microsoft Learn Documentation. Available at: <https://learn.microsoft.com>.